

Class 09

Transactions, Commit, Rollback & Automation

◆ 1. What is a Transaction?

A **Transaction** is a **single logical unit of work** that contains one or more PostgreSQL statements executed together.

It ensures that **either all operations succeed** or **none of them do** — maintaining *data consistency* and *integrity*.

Think of it as:

| A “bank transfer” — if money is deducted from one account, it must be credited to another.
| If any part fails, the whole operation is undone.

Example:

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 1000 WHERE account_id = 2;
```

```
COMMIT;
```

◆ 2. Transaction Properties — The ACID Rule

Property	Meaning	Explanation
A - Atomicity	All or nothing	Transaction fully completes or fully fails.
C - Consistency	Valid data	Database must remain valid before & after the transaction.
I - Isolation	Independent	Transactions don't interfere with each other.
D - Durability	Permanent	Once committed, changes are saved permanently.

◆ 3. COMMIT and ROLLBACK

These are **Transaction Control Commands** (TCL) used to save or undo changes.

COMMIT

Permanently saves all the changes made in the current transaction.

```
COMMIT;
```

ROLLBACK

Undo all the changes made in the current transaction (reverts database to previous stable state).

```
ROLLBACK;
```

Example:

```
START TRANSACTION;  
  
UPDATE students SET marks = marks + 5;  
  
-- Suppose something goes wrong  
  
ROLLBACK;
```

All changes are undone — marks remain unchanged.

SAVEPOINT

Used to create a **checkpoint** within a transaction — so you can roll back to that specific point instead of the whole transaction.

```
START TRANSACTION;  
  
UPDATE employees SET salary = salary + 5000 WHERE dept = 'HR';  
SAVEPOINT hr_update;  
  
UPDATE employees SET salary = salary + 8000 WHERE dept = 'Sales';  
  
ROLLBACK TO hr_update;  
  
COMMIT;
```

◆ 4. Error Handling & Consistency

Transactions help prevent inconsistent data.

Example

If an error occurs in any one postgresQL statement during a transaction:

- Changes are **not saved**.
- You can **ROLLBACK** safely

```
START TRANSACTION;  
  
UPDATE stock SET quantity = quantity - 5 WHERE product_id = 1;  
UPDATE orders SET total = 0 WHERE order_id = 501; -- error occurs  
  
ROLLBACK;
```

Both updates are undone to maintain consistency.

◆ 5. Automating Transactions with Procedures or Triggers

You can integrate **Stored Procedures** or **Triggers** to handle transactions automatically.

Example Using Procedure

```
CREATE PROCEDURE transferAmount(  
  IN sender INT, IN receiver INT, IN amt DECIMAL(10,2)  
)  
BEGIN  
  START TRANSACTION;  
  
  UPDATE accounts SET balance = balance - amt WHERE id = sender;  
  UPDATE accounts SET balance = balance + amt WHERE id = receiver;  
  
  IF amt > 0 THEN  
    COMMIT;  
  ELSE  
    ROLLBACK;  
  END IF;  
END;
```

◆ 6. Practical Use Cases of Transactions

Banking Systems – Transfer, withdrawals, deposits

Order Processing – Deduct stock, update order, update invoice

Inventory Management – Prevent double stock updates

Payroll Systems – Salary generation for multiple employees